



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Organización y Arquitectura de Computadoras

Proyecto 1. Calculadora con seno y coseno por series de Taylor en ensamblador

Grupo 1

Ortega de la Paz Rafael 420054085

19 de octubre de 2026

Objetivo

Implementar en lenguaje ensamblador, con la unidad de punto flotante de la Raspberry Pi Pico 2, una calculadora que realice las operaciones básicas (suma, resta, multiplicación, división y raíz cuadrada) y que calcule seno, coseno y tangente. El seno y el coseno se obtienen mediante series de Taylor, con los factoriales precalculados para evaluar la serie solo con multiplicaciones y sumas.

Introducción

La Raspberry Pi Pico 2 utiliza el microcontrolador RP2350, que integra dos núcleos Arm Cortex-M33 a 150 MHz. A diferencia de la Pico original (RP2040, núcleos Cortex-M0+), cada núcleo Cortex-M33 incluye una unidad de punto flotante (FPU) de precisión simple con 32 registros de 32 bits, S0 a S31, que ejecuta en hardware la suma, la resta, la multiplicación, la división y la raíz cuadrada. La FPU no ofrece funciones trascendentes como seno, coseno o logaritmo; esas deben construirse a partir de las operaciones básicas.

MicroPython permite escribir funciones en ensamblador Thumb dentro de un programa en Python mediante el decorador *@micropython.asm_thumb*. Los primeros cuatro argumentos de la función llegan en los registros r0 a r3 y el valor de regreso se toma de r0. Cuando el argumento es un arreglo (*array*), el registro recibe la dirección de su primer elemento, de modo que el ensamblador puede leer y escribir la memoria del arreglo con instrucciones de carga y almacenamiento (ldr/str para enteros, vldr/vstr para flotantes). Como la memoria se direcciona por bytes y cada dato de 32 bits ocupa cuatro, el desplazamiento del elemento i es $4 \cdot i$.

Las funciones seno y coseno pueden aproximarse con sus series de Taylor alrededor de cero, con x en radianes:

$$\operatorname{sen}(x) = x - x^3/3! + x^5/5! - x^7/7! + x^9/9! - \dots$$

$$\cos(x) = 1 - x^2/2! + x^4/4! - x^6/6! + x^8/8! - \dots$$

El seno solo tiene potencias y factoriales impares; el coseno, pares. Al cortar la serie, el error queda acotado por el primer término que se omite, que es pequeño cuando $|x|$ es pequeño. Siguiendo la recomendación del profesor, los factoriales se precalculan como sus recíprocos (1/3! = 0.1666667, 1/5! = 0.0083333, …). Así, dentro de la serie no se divide, porque la división es la operación más lenta del procesador, mientras que en la FPU de la Pico 2 la multiplicación se ejecuta prácticamente en un ciclo. También, como

pidió el profesor, se usa un número reducido de términos: hasta 9! para el seno y hasta 8! para el coseno.

Desarrollo

Operaciones básicas en la FPU

La rutina *fpu* recibe en r0 la dirección de un arreglo de flotantes [a, b, resultado] y en r1 un código de operación. Carga *a* y *b* en s0 y s1 con *vldr* y, mediante comparaciones (*cmp*) y saltos condicionales (*beq*), ejecuta una sola instrucción de la FPU según el código (Tabla 1). Al final guarda s2 con *vstr*.

Tabla 1. Códigos de operación de la rutina fpu.

Código	Operación	Instrucción
0	a + b	vadd
1	a − b	vsub
2	a × b	vmul
3	a ÷ b	vdiv
4	√a	vsqrt

Seno y coseno por Taylor con el método de Horner

Con $u = x^2$, ambas series se reescriben de forma anidada (método de Horner), con los coeficientes ya multiplicados por su signo:

$$\text{sen}(x) = x\cdot(1 + u\cdot(-1/3! + u\cdot(1/5! + u\cdot(-1/7! + u\cdot 1/9!))))$$

$$\cos(x) = 1 + u\cdot(-1/2! + u\cdot(1/4! + u\cdot(-1/6! + u\cdot 1/8!)))$$

Cada nivel del anidamiento es una multiplicación seguida de una suma (*vmul* y *vadd*). La rutina *taylor* calcula ambas funciones con 10 multiplicaciones en total, contra 16 si cada potencia de *x* se calculara por separado, y acumula menos error de redondeo. Los diez coeficientes se guardan en un arreglo *K*: los del seno en los desplazamientos 0 a 16 y los del coseno en 20 a 36.

Reducción del ángulo

La serie truncada solo es precisa para ángulos pequeños. Por eso, antes de llamar al ensamblador, el ángulo se descompone como $x = n\cdot\pi/2 + r$, con r en $[-\pi/4, \pi/4]$, y el resultado se acomoda según el cuadrante $q = n \bmod 4$ con las identidades de la Tabla 2.

Tabla 2. Identidades de reducción por cuadrante.

q	sen(x)	cos(x)
0	sen(r)	cos(r)
1	cos(r)	−sen(r)
2	−sen(r)	−cos(r)
3	−cos(r)	sen(r)

Con $|r| \le \pi/4$, el primer término omitido vale como máximo $(\pi/4)^{11}/11! \approx 1.8\times10^{-9}$ para el seno y $(\pi/4)^{10}/10! \approx 2.5\times10^{-8}$ para el coseno. Ambos son menores que la resolución de un flotante de 32 bits, que es de unos 6×10^{-8} para valores entre 0.5 y 1, así que el error de la serie queda por debajo de lo que la FPU puede representar.

Esta reducción fue una corrección que surgió en la revisión. La primera versión llevaba el ángulo solo a $[-\pi/2, \pi/2]$, y en el extremo, a 90°, la serie truncada daba sen(90°) = 1.0000034, un valor mayor que 1 que el seno nunca puede tomar. Con la reducción a $[-\pi/4, \pi/4]$, sen(90°) se calcula como cos(0) y da exactamente 1.

Tangente e interfaz de la calculadora

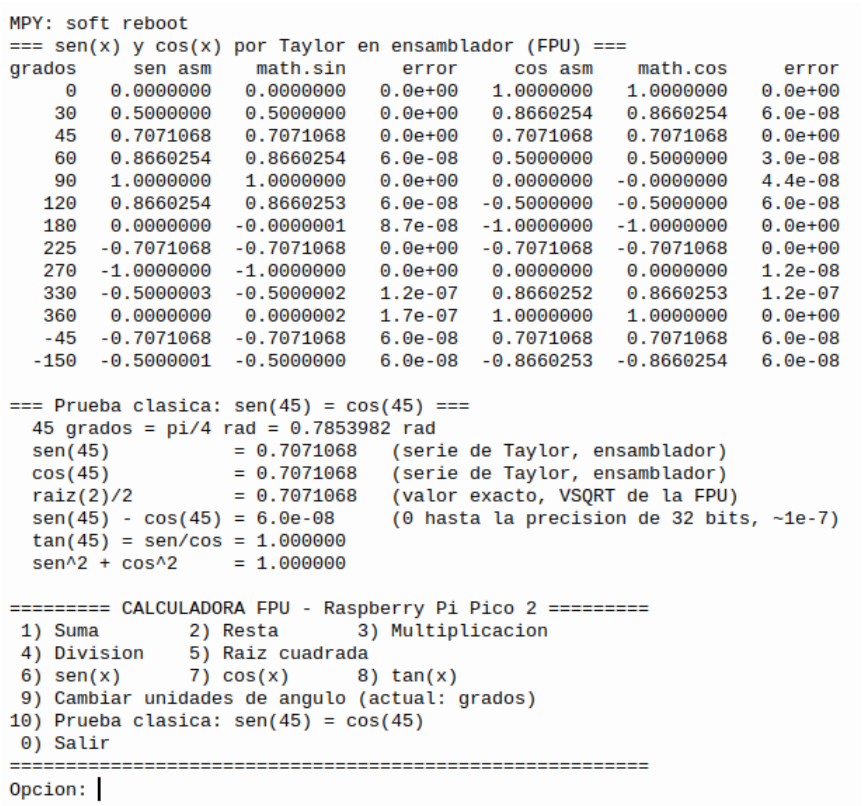
La tangente se obtiene como sen(*x*)/cos(*x*) con una sola división (*vdiv*). Cuando $|\cos(x)| < 10^{-4}$, el programa la reporta como indefinida, por ejemplo a 90°. La interfaz es un menú en la consola de Thonny: el usuario elige la operación, escribe los datos y el resultado se imprime de inmediato. El ángulo puede darse en grados o en radianes (opción 9). Python solo maneja el menú, la lectura de datos y la impresión; todas las operaciones aritméticas se hacen en ensamblador.

Prueba clásica: sen(45°) = cos(45°)

A 45° (π/4 rad) el triángulo rectángulo es isósceles: los catetos opuesto y adyacente miden lo mismo, de modo que $\sen(45°) = \cos(45°) = 1/\sqrt{2} = \sqrt{2}/2 \approx 0.7071068$. En el círculo unitario corresponde al punto $(\sqrt{2}/2, \sqrt{2}/2)$, donde $x = y$. El programa verifica esta igualdad de varias formas, todas calculadas en la FPU: compara el seno y el coseno con √2/2 obtenido con *vsqrt* y *vdiv*, calcula su diferencia con *vsub*, la tangente (que debe ser 1) y la identidad $\sen^2 + \cos^2 = 1$ con *vmul* y *vadd*.

Pruebas y resultados

Al ejecutarse, el programa imprime primero una tabla con el seno y el coseno de trece ángulos, desde −150° hasta 360°, comparados con las funciones de la biblioteca *math*. Después imprime la prueba clásica y abre el menú de la calculadora. Las Figuras 1 y 2 muestran la ejecución en la Pico 2.



```
MPY: soft reboot
=== sen(x) y cos(x) por Taylor en ensamblador (FPU) ===
grados     sen asm    math.sin     error     cos asm    math.cos     error
     0   0.0000000   0.0000000   0.0e+00   1.0000000   1.0000000   0.0e+00
    30   0.5000000   0.5000000   0.0e+00   0.8660254   0.8660254   6.0e-08
    45   0.7071068   0.7071068   0.0e+00   0.7071068   0.7071068   0.0e+00
    60   0.8660254   0.8660254   6.0e-08   0.5000000   0.5000000   3.0e-08
    90   1.0000000   1.0000000   0.0e+00   0.0000000  -0.0000000   4.4e-08
   120   0.8660254   0.8660253   6.0e-08  -0.5000000  -0.5000000   6.0e-08
   180   0.0000000  -0.0000001   8.7e-08  -1.0000000  -1.0000000   0.0e+00
   225  -0.7071068  -0.7071068   0.0e+00  -0.7071068  -0.7071068   0.0e+00
   270  -1.0000000  -1.0000000   0.0e+00   0.0000000   0.0000000   1.2e-08
   330  -0.5000003  -0.5000002   1.2e-07   0.8660252   0.8660253   1.2e-07
   360   0.0000000   0.0000002   1.7e-07   1.0000000   1.0000000   0.0e+00
   -45  -0.7071068  -0.7071068   6.0e-08   0.7071068   0.7071068   6.0e-08
  -150  -0.5000001  -0.5000000   6.0e-08  -0.8660253  -0.8660254   6.0e-08

=== Prueba clasica: sen(45) = cos(45) ===
  45 grados = pi/4 rad = 0.7853982 rad
  sen(45)           = 0.7071068   (serie de Taylor, ensamblador)
  cos(45)           = 0.7071068   (serie de Taylor, ensamblador)
  raiz(2)/2         = 0.7071068   (valor exacto, VSQRT de la FPU)
  sen(45) - cos(45) = 6.0e-08     (0 hasta la precision de 32 bits, ~1e-7)
  tan(45) = sen/cos = 1.000000
  sen^2 + cos^2     = 1.000000

========= CALCULADORA FPU - Raspberry Pi Pico 2 =========
 1) Suma        2) Resta       3) Multiplicacion
 4) Division    5) Raiz cuadrada
 6) sen(x)      7) cos(x)      8) tan(x)
 9) Cambiar unidades de angulo (actual: grados)
10) Prueba clasica: sen(45) = cos(45)
 0) Salir
==========================================================
Opcion:
```

Figura 1. Tabla de comprobación y prueba clásica sen(45°) = cos(45°) en la Pico 2.

```
========= CALCULADORA FPU - Raspberry Pi Pico 2 =========
 1) Suma        2) Resta       3) Multiplicacion
 4) Division    5) Raiz cuadrada
 6) sen(x)      7) cos(x)      8) tan(x)
 9) Cambiar unidades de angulo (actual: grados)
10) Prueba clasica: sen(45) = cos(45)
 0) Salir
=========================================================
Opcion: 6
x (grados) = 90
  sen(90.0) = 1.0000000

========= CALCULADORA FPU - Raspberry Pi Pico 2 =========
 1) Suma        2) Resta       3) Multiplicacion
 4) Division    5) Raiz cuadrada
 6) sen(x)      7) cos(x)      8) tan(x)
 9) Cambiar unidades de angulo (actual: grados)
10) Prueba clasica: sen(45) = cos(45)
 0) Salir
=========================================================
Opcion: 4
a = 9
b = 2
  9.0 / 2.0 = 4.5

========= CALCULADORA FPU - Raspberry Pi Pico 2 =========
 1) Suma        2) Resta       3) Multiplicacion
 4) Division    5) Raiz cuadrada
 6) sen(x)      7) cos(x)      8) tan(x)
 9) Cambiar unidades de angulo (actual: grados)
10) Prueba clasica: sen(45) = cos(45)
 0) Salir
=========================================================
```

Figura 2. Uso de la calculadora en la Pico 2.

En la tabla, el seno y el coseno coinciden con *math* en los siete decimales mostrados, incluidos los valores notables: sen(30°) = 0.5, sen(90°) = 1, cos(180°) = −1. En la prueba clásica, sen(45°) y cos(45°) dan 0.7071068, igual que √2/2, y la tangente y sen^2 + $\cos^2$ dan 1. La diferencia sen(45°) − cos(45°) es de 6×10^{-8}, que equivale a un solo escalón de representación (2^{-24}) de un flotante de 32 bits en ese intervalo. Es decir, ambos valores son iguales dentro de la precisión de la FPU.

Conclusiones

Se construyó una calculadora en ensamblador que aprovecha la FPU de la Pico 2 para las operaciones básicas y que obtiene seno, coseno y tangente, funciones que el hardware no tiene, a partir de series de Taylor. Precalcular los recíprocos de los factoriales elimina las divisiones dentro de la serie, y el método de Horner reduce las multiplicaciones necesarias. El caso de sen(90°) mostró que el número de términos no basta por sí solo: la exactitud también depende del rango en que se evalúa la serie. Al

reducir el ángulo a ±45° y usar identidades trigonométricas, la serie truncada alcanzó la precisión máxima de un flotante de 32 bits, como confirma la prueba sen(45°) = cos(45°).

Bibliografía

Arm Limited. (2023). *Armv8-M architecture reference manual* (DDI0553). Arm Limited.

Arredondo Garza, J. A. de J. (2026). *Organización y Arquitectura de Computadoras: clases del semestre 2027-1* [Exposiciones en clase]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

Burden, R. L., Faires, J. D., & Burden, A. M. (2016). *Numerical analysis* (10.ª ed.). Cengage Learning.

IEEE. (2019). *IEEE standard for floating-point arithmetic* (IEEE Std 754-2019). Institute of Electrical and Electronics Engineers.

MicroPython. (s. f.). *Inline assembler for Thumb2 architectures*. Recuperado el 19 de octubre de 2026 de https://docs.micropython.org/en/latest/reference/asm_thumb2_index.html

Raspberry Pi Ltd. (2025). *RP2350 datasheet: A microcontroller by Raspberry Pi*. https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf

Anexo: código fuente (proyecto_calculadora.py)

```
# Proyecto: Calculadora con la FPU de la Raspberry Pi Pico 2 (RP2350)
# sen(x) y cos(x) por series de Taylor + operaciones basicas.
# No corre en Pico 1 (sin FPU).
# Toda la aritmetica se hace en ensamblador (MicroPython inline Thumb):
#   - sen(x) y cos(x): series de Taylor evaluadas con Horner, solo multiplicaciones y sumas
#   - Suma, resta, multiplicacion, division y raiz: instrucciones de la FPU
#   - tan(x) = sen(x) / cos(x) con VDIV
# Python solo maneja el menu, la lectura de datos y la impresion.
from array import array
import math

# ---------------- Operaciones básicas en la FPU ----------------
# r0 -> array('f', [a, b, resultado]);  r1 = operación
#   0: a + b   1: a - b   2: a * b   3: a / b   4: raíz(a)
@micropython.asm_thumb
def fpu(r0, r1):
    vldr(s0, [r0, 0])     # s0 = a
    vldr(s1, [r0, 4])     # s1 = b
    cmp(r1, 0)
    beq(SUMA)
    cmp(r1, 1)
    beq(RESTA)
    cmp(r1, 2)
    beq(MULT)
    cmp(r1, 3)
    beq(DIVI)
    vsqrt(s2, s0)         # 4: raíz cuadrada
    b(FIN)
    label(SUMA)
    vadd(s2, s0, s1)
    b(FIN)
    label(RESTA)
    vsub(s2, s0, s1)
    b(FIN)
    label(MULT)
    vmul(s2, s0, s1)
    b(FIN)
    label(DIVI)
    vdiv(s2, s0, s1)
    label(FIN)
    vstr(s2, [r0, 8])     # resultado

# ---------------- sen(x) y cos(x) por Taylor (metodo de Horner) ----------------
# sen(x) = x(1 - u/3! + u^2/5! - u^3/7! + u^4/9!)        con u = x^2
# cos(x) =    1 - u/2! + u^2/4! - u^3/6! + u^4/8!
# Horner: el polinomio se evalua de adentro hacia afuera, t = t*u + c,
# asi se usan 5 multiplicaciones por funcion en lugar de calcular cada potencia.
# Coeficientes con signo = reciprocos de los factoriales (precalculados, sin dividir).
K = array('f', [1.0, -1 / 6, 1 / 120, -1 / 5040, 1 / 362880,      # seno:   offsets 0..16
                1.0, -1 / 2, 1 / 24, -1 / 720, 1 / 40320])        # coseno: offsets 20..36

# r0 -> array('f',[x]), r1 -> array('f',[sen, cos]), r2 -> K
```

```
@micropython.asm_thumb
def taylor(r0, r1, r2):
    vldr(s0, [r0, 0])     # s0 = x
    vmul(s1, s0, s0)      # s1 = u = x^2
    # ---- seno ----
    vldr(s2, [r2, 16])    # t = 1/9!
    vldr(s3, [r2, 12])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # t = t*u - 1/7!
    vldr(s3, [r2, 8])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # t = t*u + 1/5!
    vldr(s3, [r2, 4])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # t = t*u - 1/3!
    vldr(s3, [r2, 0])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # t = t*u + 1
    vmul(s2, s2, s0)      # sen = t*x
    vstr(s2, [r1, 0])
    # ---- coseno ----
    vldr(s2, [r2, 36])    # t = 1/8!
    vldr(s3, [r2, 32])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # t = t*u - 1/6!
    vldr(s3, [r2, 28])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # t = t*u + 1/4!
    vldr(s3, [r2, 24])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # t = t*u - 1/2!
    vldr(s3, [r2, 20])
    vmul(s2, s2, s1)
    vadd(s2, s2, s3)      # cos = t*u + 1
    vstr(s2, [r1, 4])

MEDIO_PI = math.pi / 2

def reducir(x):
    # x = n*(pi/2) + r, con r en [-pi/4, pi/4]: ahi la serie es practicamente exacta.
    # q = n mod 4 indica el cuadrante para acomodar sen y cos al final.
    n = math.floor(x / MEDIO_PI + 0.5)
    r = x - n * MEDIO_PI
    return r, int(n) % 4

# ---------------- Funciones de la calculadora ----------------
_ab = array('f', [0.0, 0.0, 0.0])
_x = array('f', [0.0])
_sc = array('f', [0.0, 0.0])

def op(a, b, codigo):
    _ab[0] = a
    _ab[1] = b
    fpu(_ab, codigo)
```

```
    return _ab[2]

def sen_cos(x):
    r, q = reducir(x)
    _x[0] = r
    taylor(_x, _sc, K)
    s, c = _sc[0], _sc[1]
    if q == 1:
        s, c = c, -s      # sen(r + 90) = cos(r),   cos(r + 90) = -sen(r)
    elif q == 2:
        s, c = -s, -c     # sen(r + 180) = -sen(r), cos(r + 180) = -cos(r)
    elif q == 3:
        s, c = -c, s      # sen(r + 270) = -cos(r), cos(r + 270) = sen(r)
    return s + 0.0, c + 0.0   # + 0.0 evita imprimir "-0.0000000"

def leer(msg):
    while True:
        try:
            return float(input(msg))
        except ValueError:
            print("  Escribe un numero (usa punto decimal)")

# ---------------- Prueba clasica: sen(45) = cos(45) ----------------
# A 45 grados (pi/4 rad) el triangulo rectangulo es isosceles: los dos catetos
# miden lo mismo, asi que sen(45) = cos(45) = 1/raiz(2) = raiz(2)/2 = 0.7071068.
# Todas las cuentas de esta prueba se hacen en la FPU (ensamblador).
def prueba_clasica():
    x = math.radians(45)
    s, c = sen_cos(x)
    r2_2 = op(op(2.0, 0.0, 4), 2.0, 3)              # raiz(2) / 2   (VSQRT y VDIV)
    dif = op(s, c, 1)                                # sen - cos     (VSUB)
    tan = op(s, c, 3)                                # sen / cos     (VDIV)
    pit = op(op(s, s, 2), op(c, c, 2), 0)            # sen^2 + cos^2 (VMUL y VADD)
    print()
    print("=== Prueba clasica: sen(45) = cos(45) ===")
    print("  45 grados = pi/4 rad = {:.7f} rad".format(x))
    print("  sen(45)           = {:.7f}   (serie de Taylor, ensamblador)".format(s))
    print("  cos(45)           = {:.7f}   (serie de Taylor, ensamblador)".format(c))
    print("  raiz(2)/2         = {:.7f}   (valor exacto, VSQRT de la FPU)".format(r2_2))
    print("  sen(45) - cos(45) = {:.1e}     (0 hasta la precision de 32 bits, ~1e-
7)".format(abs(dif)))
    print("  tan(45) = sen/cos = {:.6f}".format(tan))
    print("  sen^2 + cos^2     = {:.6f}".format(pit))

# ---------------- Prueba: tabla de sen y cos ----------------
print("=== sen(x) y cos(x) por Taylor en ensamblador (FPU) ===")
print("{:>6} {:>11} {:>11} {:>9} {:>11} {:>11} {:>9}".format(
    "grados", "sen asm", "math.sin", "error", "cos asm", "math.cos", "error"))
for g in (0, 30, 45, 60, 90, 120, 180, 225, 270, 330, 360, -45, -150):
    x = math.radians(g)
    s, c = sen_cos(x)
    print("{:>6} {:>11.7f} {:>11.7f} {:>9.1e} {:>11.7f} {:>11.7f} {:>9.1e}".format(
        g, s, math.sin(x), abs(s - math.sin(x)), c, math.cos(x), abs(c - math.cos(x))))
```

```
prueba_clasica()

# ---------------- Calculadora ----------------
grados = True

MENU = """
========= CALCULADORA FPU - Raspberry Pi Pico 2 =========
 1) Suma        2) Resta       3) Multiplicacion
 4) Division    5) Raiz cuadrada
 6) sen(x)      7) cos(x)      8) tan(x)
 9) Cambiar unidades de angulo (actual: {})
10) Prueba clasica: sen(45) = cos(45)
 0) Salir
============================================================"""

while True:
    print(MENU.format("grados" if grados else "radianes"))
    opcion = input("Opcion: ").strip()

    if opcion == "0":
        print("Fin.")
        break

    elif opcion in ("1", "2", "3", "4"):
        a = leer("a = ")
        b = leer("b = ")
        simbolo = "+-*/"[int(opcion) - 1]
        if opcion == "4" and b == 0:
            print("  Error: division entre cero")
            continue
        r = op(a, b, int(opcion) - 1)
        print("  {} {} {} = {:.7g}".format(a, simbolo, b, r))

    elif opcion == "5":
        a = leer("a = ")
        if a < 0:
            print("  Error: la FPU no admite raiz de negativos")
            continue
        print("  raiz({}) = {:.7g}".format(a, op(a, 0.0, 4)))

    elif opcion in ("6", "7", "8"):
        ang = leer("x ({}) = ".format("grados" if grados else "radianes"))
        x = math.radians(ang) if grados else ang
        s, c = sen_cos(x)
        if opcion == "6":
            print("  sen({}) = {:.7f}".format(ang, s))
        elif opcion == "7":
            print("  cos({}) = {:.7f}".format(ang, c))
        else:
            if abs(c) < 1e-4:
                print("  tan({}) = indefinida (cos = 0)".format(ang))
            else:
                print("  tan({}) = {:.6f}".format(ang, op(s, c, 3)))
```

```
    elif opcion == "10":
        prueba_clasica()

    elif opcion == "9":
        grados = not grados
        print("  Ahora en", "grados" if grados else "radianes")

    else:
        print("  Opcion no valida")
```